

Secure Chat Application

Submitted By

Faizan Ali
22i-2496

Hamail Rehman
22k-4443

Department of Computer Science

National University of Computer & Emerging Sciences

Project Repo :

<https://github.com/FaizanAli-io/Secure-Chat-Application.git>

1. Motivation

In an era where data privacy is increasingly critical, the motivation for this project stems from the need to ensure secure, real-time communication for users over the internet. With the growing demand for secure messaging applications, this project aims to offer a practical, encrypted messaging solution while providing hands-on experience with network

*protocols, frontend-backend integration, and
cryptographic principles.*

2. Overview

2.1 Significance of the Project

The Secure Chat Application is significant due to its practicality and relevance in the modern digital world, where communication is frequent and often sensitive. It demonstrates how encrypted chat systems can ensure privacy, reduce cyber threats, and give users control over their data. The academic value lies in its integration of networking, cryptography, and full-stack web development concepts.

2.2 Description of the Project

This project is a secure, real-time chat platform that enables users to create and join chat rooms, send encrypted messages, and authenticate securely. It features AES-based end-to-end encryption on the frontend using CryptoJS, ensuring that even the server does not have access to plaintext messages. The application supports user registration/login, chat room management, and secure communication channels using React, Node.js, and MongoDB.

2.3 Background of the Project

The development of this application draws from extensive documentation on the following tools and concepts:

- AES encryption and secure key handling (CryptoJS GitHub)
- React.js framework for dynamic user interfaces
- Express.js and Node.js for server-side development
- Socket.io for real-time data exchange

2.4 Project Category

This project falls under the Product-Based category, aiming to deliver a usable and secure communication solution.

3. Features / Scope / Modules

1. End-to-End Encrypted Messaging

Messages are encrypted on the client-side using AES and decrypted only by the recipient, ensuring confidentiality.

2. User Authentication

Users can register and log in securely using JWT-based authentication tokens.

3. Real-Time Chat Rooms

Users can create and join chat rooms to exchange encrypted messages in real time.

4. Responsive Frontend UI

Built with React.js and Tailwind CSS, offering an intuitive and mobile-friendly interface.

5. Message Routing and Storage

Server receives and routes encrypted messages and stores them in a secure database (MongoDB/SQLite).

6. Modular and Scalable Architecture

Backend and frontend are separated, promoting scalability and maintainability.

7. Secure Key Handling

Although a static key is currently used for demo purposes, future versions plan to adopt environment-variable-based secure key exchange.

4. Project Planning

Project planning and responsibilities:

Week	Task Description	Responsible Member
1	Requirements gathering & tech stack finalization	Faizan & Hamail
2-3	Frontend (React setup, UI design)	Faizan & Hamail
4-5	Backend APIs and authentication	Faizan
6	Message encryption/decryption integration	Hamail
7	Real-time messaging setup with Socket.io	Faizan
8	Testing and debugging	Both
9	Final documentation and presentation prep	Both

5. Project Feasibility

Technical Feasibility

The application is feasible using widely adopted tools (React, Node.js, MongoDB, CryptoJS). The real-time architecture is implemented with standard WebSockets (via Socket.io).

Economic Feasibility

The project uses open-source tools, so there are no licensing or platform fees. Hosting costs for deployment can be managed with free or low-cost tiers (e.g., Vercel, Heroku).

Schedule Feasibility

Given a 9-week development timeline, the scope was divided effectively, with major milestones aligned with the course schedule.

6. Hardware and Software Requirements

Software

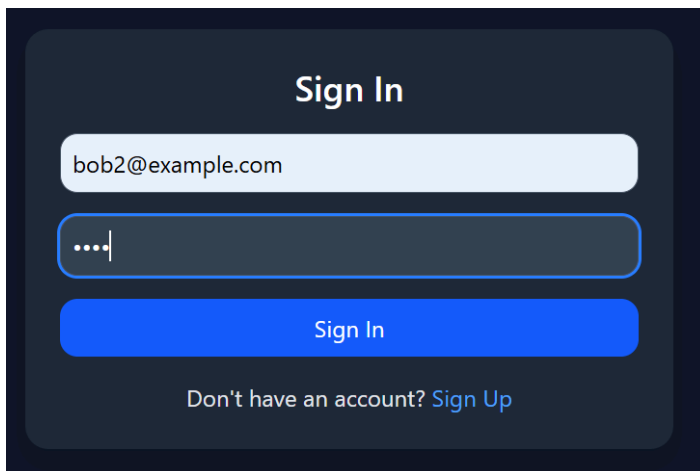
- Node.js (v18+)
- React.js
- MongoDB or SQLite
- VS Code / any text editor
- Git and GitHub
- Postman for API testing

Hardware

- Any modern PC/laptop with at least:
 - 4 GB RAM
 - Dual-core processor
 - Internet connection

7. Diagrammatic Representation of the Overall System

UI :



Create Room

Room name

☐ Faizan Ali
 ☐ Hamail Rehman
 ☐ Test User
 ☐ Bob

Cancel

Create

Rooms

Group_chat-05

View Participants

10:46 PM Wassup

Bob

All good wby

10:48 PM

+ Create Room

Logout

Type a message...

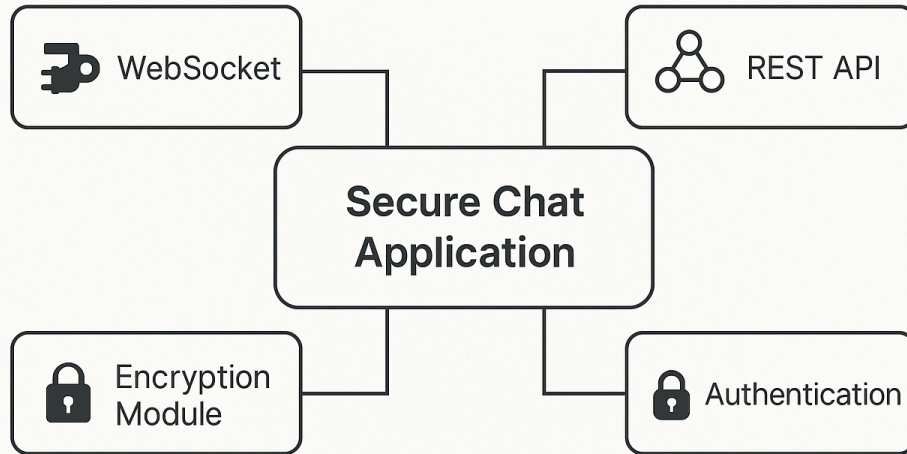
Send

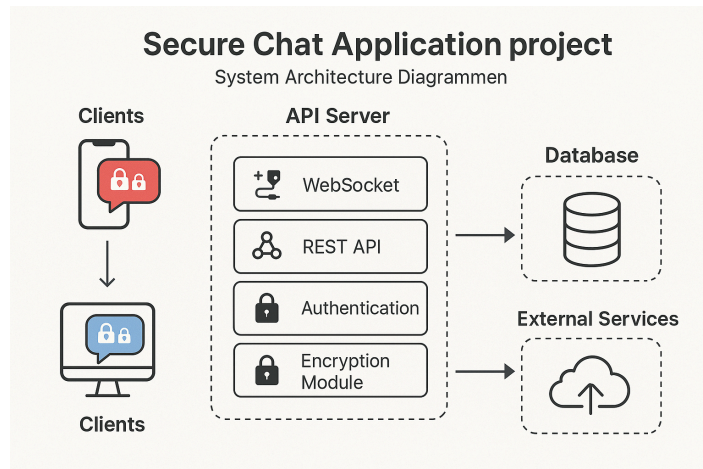
Database :

[User] [RoomUser] [Room] [Message] [Close all]									
[Filters] [None] [Fields] [All] [Showing 5 of 5] [Add record] [Save 1 change] [Discard changes]									
	id	email	createdAt	password	updatedAt	username	authKey	messages	rooms
	49830eb0-9f1d-44b1-819c...	bob@example.com	2025-05-04T16:52:21.450Z	\$2b\$10\$cLhO.57zgTVzIXTc...	2025-05-04T16:52:21.450Z	Bob	a6bc38ed467eec44fc5997c...	2 Message	2 RoomUser
	9796fd7c-8adb-4132-ab2c...	faizan@example.com	2025-05-04T09:01:29.487Z	\$2b\$10\$vb71xz8PGGfydzqL...	2025-05-04T09:01:29.487Z	Faizan Ali	02c4ad4175d6db7ca9d51b9...	2 Message	3 RoomUser
	cc66ae87-98b9-4a15-96aa...	hamail@example.com	2025-05-04T09:01:53.618Z	\$2b\$10\$8Ym.H0Xivn8EuPcH...	2025-05-04T09:01:53.618Z	Hamail Rehman	1c11d5a016683ea58c628cb...	3 Message	4 RoomUser
	d013b0b2-d852-4990-aa33...	bob@example.com	2025-05-04T16:54:22.425Z	\$2b\$10\$VmdOF.sRmc/vtBb...	2025-05-04T16:54:22.425Z	Bob2	324ba43e6d6ad7d29b5f921...	1 Message	1 RoomUser
	d6baafb8-58c4-4359-908d...	test@example.com	2025-05-04T09:57:59.808Z	\$2b\$10\$Dvv29zrMl1Ns0B8J...	2025-05-04T09:57:59.808Z	Test User	d38509eb985d4fcf97c3ba...	3 Message	1 RoomUser

Secure Chat Application

Component Diagram





[User] <---> [Frontend (React + CryptoJS)]
 <--encrypted msg-->
 [Backend (Express + Socket.io)]
 <--> [MongoDB / SQLite]

Figure 1. Secure Chat System Architecture

8. Test Plan & Results

Test Case ID	Description	Expected Result	Actual Result	Status
TC01	Send Encrypted Message	Decryption Successful	Success	Pass
TC02	Socket Disconnection	Auto Reconnect	Success	Pass
TC03	Access Without Token	Unauthorized Error	Success	Pass
TC04	Corrupted Message	Decryption Fail Gracefully	Warning Message	Pass

Testing tools: Jest, React Testing Library, Manual Tests, Postman

9. Security Threat Model

- **Threat:** Packet sniffing during message transmission
Mitigation: End-to-end encryption with AES
 - **Threat:** Unauthorized access via localStorage tokens
Mitigation: Secure HTTPOnly JWT tokens can be considered
 - **Threat:** XSS in chat input
Mitigation: Input sanitization and encoding
 - **Threat:** Hardcoded keys
Mitigation: Handled securely using `.env` variables
-

10. Future Work

- Implement mobile-first PWA

- Add media file encryption (images, PDFs)
- Introduce multi-device session syncing
- Incorporate biometric authentication
- Explore quantum-resistant encryption methods

8. References

[1] React Documentation, <https://react.dev/>

[2] Node.js Documentation, <https://nodejs.org/>

[3] CryptoJS Documentation, <https://github.com/brix/crypto-js>

[4] Express.js Documentation, <https://expressjs.com/>

[5] Socket.io Documentation, <https://socket.io/>